

Union-Find 与集合划分

动态合并后，怎样快速回答“是否同一集合”

408 · 数据结构 Topic DS-06

母问题：动态连通不是一般树，而是代表元问题

并查集的生成链是

Partition → Representative → Find/Union → Path Compression + Rank → Dynamic Connectivity.

每个集合用一棵父指针树表示，但树形只是实现手段；真正的抽象是分区。find(x) 返回集合代表元，union(a,b) 合并两个代表元不同的集合。任何操作后，父指针必须最终通向唯一根，根的数量必须等于集合数量。

本册定位与 Owner 边界

- **Owns:** MakeSet、Find、Union、代表元、按秩/大小合并、路径压缩与动态连通语义。
- **Uses:** 数组下标和成本向量；向 DS-B03/DS08 输出 ‘connected/unite’ 接口。
- **Stops:** 一般树遍历、Kruskal 的边权排序与生成树正确性由其他 Owner 负责。

目录

1	分区不变量	2
2	为什么需要两种局部修复	2
3	代码：代表元、路径压缩与按秩合并	2
3.1	测试：重复合并、压缩与空全集	4
4	复杂度与边界	5
5	做题控制协议与压缩复原	5

1 分区不变量

初始时每个元素自成集合, $parent[i] = i$ 。父指针反复追踪必到根; 根是自己的父节点。若两个元素根相同, 则属于同一集合; 若根不同, 合并时把一个根挂到另一个根下, 并将集合数减一。重复合并不改变分区。

2 为什么需要两种局部修复

若只把一个根挂到另一个根下, 连续合并可能生成长链, 使 Find 退化到 $O(n)$ 。按秩合并总把较矮树挂到较高树, 限制树高; 路径压缩在 Find 返回时让沿途节点直接指向根, 摊还复杂度达到近似常数 $O(\alpha(n))$ 。

3 代码: 代表元、路径压缩与按秩合并

完整实现位于 code/ds06_union_find.hpp, 测试位于 code/ds06_union_find_test.cpp; 正文直接引用真实源码。

Listing 1: Find 的路径压缩与 Union 的按秩合并

```
11 public:
12     explicit UnionFind(std::size_t count)
13         : parent_(count), rank_(count, 0), components_(count) {
14         for (std::size_t i = 0; i < count; ++i) {
15             parent_[i] = i;
16         }
17     }
18
19     std::size_t size() const { return parent_.size(); }
20     std::size_t components() const { return components_; }
21
22     std::size_t find(std::size_t value) {
23         check(value);
24         if (parent_[value] != value) {
25             parent_[value] = find(parent_[value]);
26         }
27         return parent_[value];
28     }
29
30     bool connected(std::size_t left, std::size_t right) {
31         return find(left) == find(right);
32     }
33
34     bool unite(std::size_t left, std::size_t right) {
35         std::size_t root_left = find(left);
36         std::size_t root_right = find(right);
37         if (root_left == root_right) {
38             return false;
39         }
```

```

40     if (rank_[root_left] < rank_[root_right]) {
41         std::swap(root_left, root_right);
42     }
43     parent_[root_right] = root_left;
44     if (rank_[root_left] == rank_[root_right]) {
45         ++rank_[root_left];
46     }
47     --components_;
48     return true;
49 }
50
51 bool invariant() const {
52     if (components_ > parent_.size() || parent_.size() != rank_.size()) {
53         return false;
54     }
55     std::size_t roots = 0;
56     for (std::size_t i = 0; i < parent_.size(); ++i) {
57         if (parent_[i] >= parent_.size() || rank_[i] > parent_.size()) {
58             return false;

```

‘find’ 的递归返回值就是代表元；回溯赋值是路径压缩。‘unite’ 先分别 Find，再比较秩，最后只减少一次 ‘components’。根相同的分支必须提前返回，否则重复合并会错误减少集合数。

Listing 2: 分区不变量和越界边界

```

60     if (parent_[i] == i) {
61         ++roots;
62     }
63 }
64 return roots == components_;
65 }
66
67 private:
68 void check(std::size_t value) const {
69     if (value >= parent_.size()) {
70         throw std::out_of_range("union-find element out of range");
71     }
72 }
73
74 std::vector<std::size_t> parent_;
75 std::vector<std::size_t> rank_;
76 std::size_t components_ = 0;
77 };
78
79 } // namespace ipara::ds06
80
81 #endif

```

3.1 测试：重复合并、压缩与空全集

Listing 3: 代表元一致性和集合数变化测试

```
7
8 void test_make_set_and_bounds() {
9     UnionFind sets(4);
10    assert(sets.size() == 4 && sets.components() == 4 && sets.invariant());
11    bool threw = false;
12    try {
13        sets.find(4);
14    } catch (const std::out_of_range&) {
15        threw = true;
16    }
17    assert(threw);
18 }
19
20 void test_union_and_duplicate_union() {
21     UnionFind sets(6);
22     assert(sets.unite(0, 1));
23     assert(sets.unite(2, 3));
24     assert(sets.unite(1, 2));
25     assert(!sets.unite(0, 3));
26     assert(sets.components() == 3);
27     assert(sets.connected(0, 3));
28     assert(!sets.connected(0, 4));
29     assert(sets.invariant());
30 }
31
32 void test_path_compression_reaches_representative() {
33     UnionFind sets(8);
34     sets.unite(0, 1);
35     sets.unite(2, 3);
36     sets.unite(4, 5);
37     sets.unite(6, 7);
38     sets.unite(0, 2);
39     sets.unite(4, 6);
40     sets.unite(0, 4);
41     const std::size_t representative = sets.find(7);
42     assert(representative == sets.find(0));
43     assert(sets.invariant());
44 }
45
46 void test_empty_universe() {
47     UnionFind sets(0);
48     assert(sets.components() == 0 && sets.invariant());
49 }
50
```

```
51 int main() {
52     test_make_set_and_bounds();
}

clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \\\
code/ds06_union_find_test.cpp -o /tmp/ds06_test
/tmp/ds06_test
```

4 复杂度与边界

操作	摊还复杂度	保证	边界
Find/Union	$O(\alpha(n))$	路径压缩 + 按秩/大小	空集合、越界
MakeSet	$O(n)$	每项自为根	$n = 0$ 合法
空间	$O(n)$	parent/rank 两数组	不保存集合内部顺序

不要把“父指针树”当作业务树

并查集不支持按层遍历、输出路径或保持关键字有序；它只回答代表元和同集合关系。把 Union-Find 的父树画成一般树来做遍历，会把实现结构误当成目标对象。

5 做题控制协议与压缩复原

先写当前分区和每个根；每次合并只比较两个代表元；重复合并检查是否同根；若问复杂度，指出路径压缩和秩策略共同作用。忘掉代码后复原：集合如何表示？谁是代表元？Find 是否压缩？Union 挂哪棵树？集合数何时减少？

跨册交接

DS06 向 DS-B03 和 DS08 只提供 ‘connected/unite’。Kruskal 的排序、选边和无环性证明由 DS08 Own；本册只保证每次加入边前后动态分区不变量。

旧总册关于代表元、路径压缩、按秩合并和 Kruskal 接口的内容已吸收；带权并查集、可回滚并查集和持久化版本作为 Extension。